ERP

BASE ARCHITECTURE DOCUMENT

Documento base de arquitectura del proyecto

Architecture: Clean Architecture + Domain-Driven Design (DDD) + CQRS

Platform: .NET 10 · C# · ASP.NET Core Web API · Entity Framework Core · SQL Server

Delivery style: Modular Monolith (Monolito modular)

Versión 1.0 · 9 de octubre de 2026

Nombres de código en inglés con traducciones al español entre paréntesis.

1. Propósito y alcance

Este documento establece la arquitectura inicial de un ERP orientado a pequeños y medianos negocios. Define los proyectos, módulos de negocio, reglas de dependencia, convenciones de nombres, tecnologías y una ruta de implementación. Es una base de diseño que debe evolucionar conforme se validen los requisitos reales.

Objetivos

- Administrar empresas, usuarios, catálogos, compras, inventario, producción, ventas y reportes mediante una API.
- Mantener las reglas de negocio independientes de SQL Server, Entity Framework Core y ASP.NET Core.
- Organizar los casos de uso con CQRS, separando comandos de consultas.
- Permitir pruebas automatizadas y crecimiento funcional sin introducir microservicios prematuramente.
- Practicar inglés técnico usando nombres de clases, proyectos y carpetas en inglés.

Fuera del alcance inicial

Contabilidad completa, nómina, MRP avanzado, facturación electrónica específica por país, integración con proveedores externos y despliegue distribuido se consideran evoluciones futuras, salvo que los requisitos del negocio indiquen lo contrario.

2. Principios arquitectónicos

Principio	Aplicación en el ERP
Clean Architecture (Arquitectura Limpia)	Las dependencias del código apuntan hacia el dominio y las reglas internas.
DDD (Diseño Dirigido por el Dominio)	Modelar conceptos, invariantes y lenguaje del negocio; usar agregados donde aporten consistencia.
CQRS	Separar operaciones que modifican el estado (Commands) de las que leen datos (Queries).
Modular Monolith (Monolito modular)	Un despliegue inicial con módulos separados por responsabilidad y límites explícitos.
Dependency Inversion (Inversión de dependencias)	Application define abstracciones; Infrastructure implementa adaptadores técnicos.
Pragmatic DDD (DDD pragmático)	Aplicar patrones DDD cuando protejan reglas reales, evitando clases ceremoniales sin valor.

3. Solution structure (Estructura de la solución)

```
ERP.sln
|
+-- docs/
|   +-- Architecture/
|   +-- Database/
|   +-- Diagrams/
|   +-- UseCases/
|   +-- API/
|
+-- src/
|   +-- ERP.API/
|   +-- ERP.Application/
|   +-- ERP.Domain/
|   +-- ERP.Infrastructure/
|   +-- ERP.Contracts/
|   +-- ERP.SharedKernel/
|   +-- ERP.CrossCutting/   [opcional; evitar duplicar Infrastructure]
|
+-- tests/
    +-- ERP.Domain.Tests/
    +-- ERP.Application.Tests/
    +-- ERP.Infrastructure.Tests/
    +-- ERP.API.Tests/
    +-- ERP.Integration.Tests/
```

Nota de diseño: ERP.CrossCutting es opcional. Antes de crearlo, comprobar que sus responsabilidades no pertenecen mejor a Application o Infrastructure. Mantener proyectos separados solo cuando exista una frontera clara y útil.

4. Project responsibilities (Responsabilidades de proyectos)

ERP.Domain (Dominio)

Contiene el modelo de negocio: entidades, agregados, objetos de valor, reglas, eventos y excepciones de dominio. No referencia EF Core, ASP.NET Core, SQL Server ni proyectos externos.

ERP.Application (Aplicación)

Orquesta casos de uso, define interfaces necesarias, ejecuta validaciones de entrada y transforma resultados a modelos de respuesta. Puede utilizar MediatR si se adopta como mediador.

ERP.Infrastructure (Infraestructura)

Implementa persistencia, repositorios, integraciones externas, correo, archivos, caché y servicios técnicos. Aquí viven DbContext, configuraciones de EF Core y migraciones.

ERP.API (API)

Expone endpoints HTTP, aplica autenticación/autorización, configura middleware, traduce solicitudes a casos de uso y devuelve respuestas HTTP. No debe contener reglas de negocio.

ERP.Contracts (Contratos)

Contiene contratos públicos compartidos cuando exista una necesidad real de reutilizarlos. Para evitar dependencias innecesarias, los DTO de uso interno pueden vivir en Application y los contratos HTTP en API o en este proyecto si se comparten con clientes.

ERP.SharedKernel (Núcleo compartido)

Debe ser pequeño. Puede contener tipos realmente compartidos como Result, errores comunes o primitivas. No debe convertirse en un cajón de sastre ni crear acoplamiento entre módulos.

Tests (Pruebas)

Incluye pruebas unitarias de dominio y aplicación, pruebas de integración de persistencia y pruebas de API. Los tests deben validar comportamiento, no solo detalles internos de implementación.

5. Dependency rules (Reglas de dependencia)

```
ERP.API -----------> ERP.Application
ERP.API -----------> ERP.Infrastructure  (solo composición/arranque)
ERP.Application ---> ERP.Domain
ERP.Infrastructure -> ERP.Application    (interfaces de aplicación, si corresponde)
ERP.Infrastructure -> ERP.Domain

ERP.Domain --------> no depende de proyectos externos de la solución
```

La composición de dependencias se configura en el punto de entrada. Infrastructure puede referenciar Application para implementar interfaces definidas allí; Domain debe permanecer en el centro. Evitar referencias circulares.

6. Business modules (Módulos de negocio)

Module	Traducción	Responsabilidad principal
Security	Seguridad	Usuarios, roles, permisos y autenticación.
Companies	Empresas	Empresas, sucursales y estructura organizacional.
Catalogs	Catálogos	Clientes, proveedores, productos, unidades, marcas e impuestos.
Purchasing	Compras	Órdenes de compra y recepción de mercancías.
Inventory	Inventario	Almacenes, existencias, movimientos y ajustes.
Production	Producción	Recetas, órdenes, consumos, mermas, calidad y costos.
Sales	Ventas	Cotizaciones, pedidos, ventas, facturas y pagos.
Reports	Reportes	Consultas e indicadores de gestión.
Configuration	Configuración	Parámetros y numeración documental.

7. Domain model outline (Esquema del modelo de dominio)

Security (Seguridad)

```
Security/
  User.cs                (Usuario)
  Role.cs                (Rol)
  Permission.cs           (Permiso)
  RolePermission.cs       (Rol-Permiso)
  RefreshToken.cs         (Token de renovación)
  UserSession.cs          (Sesión de usuario)
```

Companies (Empresas)

```
Companies/
  Company.cs              (Empresa)
  Branch.cs               (Sucursal)
  Department.cs           (Departamento)
  Position.cs             (Puesto)
```

Catalogs (Catálogos)

```
Catalogs/
  Customer.cs             (Cliente)
  Supplier.cs             (Proveedor)
  Product.cs              (Producto)
  ProductCategory.cs      (Categoría de producto)
  UnitOfMeasure.cs        (Unidad de medida)
  Brand.cs                (Marca)
  Tax.cs                  (Impuesto)
```

Purchasing (Compras)

```
Purchasing/
  PurchaseOrder.cs        (Orden de compra)
  PurchaseOrderDetail.cs  (Detalle de orden de compra)
  PurchaseReceipt.cs      (Recepción de compra)
  PurchaseReceiptDetail.cs (Detalle de recepción de compra)
```

Inventory (Inventario)

```
Inventory/
  InventoryMovement.cs    (Movimiento de inventario)
  ProductStock.cs         (Existencia de producto)
  Warehouse.cs            (Almacén)
  WarehouseLocation.cs    (Ubicación de almacén)
  ProductBatch.cs         (Lote de producto)
  ProductSerial.cs        (Número de serie)
  InventoryAdjustment.cs  (Ajuste de inventario)
```

Nota: evitar una entidad genérica Inventory si representa solo la suma de existencias. Normalmente ProductStock o una proyección de existencias expresa mejor el concepto. El diseño final depende de cómo se gestionen almacenes, lotes y ubicaciones.

Production (Producción)

```
Production/
  ProductionRecipe.cs     (Receta de producción)
  RecipeDetail.cs         (Detalle de receta)
  RecipeVersion.cs        (Versión de receta)
  ProductionOrder.cs      (Orden de producción)
  ProductionOrderDetail.cs (Detalle de orden de producción)
  ProductionProcess.cs    (Proceso de producción)
  ProductionStage.cs      (Etapa de producción)
  WorkCenter.cs           (Centro de trabajo)
  Machine.cs              (Máquina)
  Operator.cs             (Operador)
  ProductionShift.cs      (Turno de producción)
  MaterialConsumption.cs  (Consumo de material)
  ProductionMovement.cs   (Movimiento de producción)
  FinishedProduct.cs      (Producto terminado)
  ProductionWaste.cs      (Merma de producción)
  ProductionCost.cs       (Costo de producción)
  QualityControl.cs       (Control de calidad)
  ProductionLog.cs        (Bitácora de producción)
```

Para una primera versión, comenzar con receta/lista de materiales, orden de producción, consumo de materiales, registro de producto terminado y merma. Añadir máquinas, turnos y control de calidad cuando el proceso real lo requiera.

Sales (Ventas)

```
Sales/
  Quotation.cs            (Cotización)
  QuotationDetail.cs      (Detalle de cotización)
  SalesOrder.cs           (Pedido de venta)
  SalesOrderDetail.cs     (Detalle de pedido de venta)
  Sale.cs                 (Venta)
  SaleDetail.cs           (Detalle de venta)
  Invoice.cs              (Factura)
  Payment.cs              (Pago)
```

Reports and Configuration (Reportes y configuración)

```
Reports/
  SalesReport.cs          (Reporte de ventas)
  InventoryReport.cs      (Reporte de inventario)
  ProductionReport.cs     (Reporte de producción)

Configuration/
  CompanyConfiguration.cs  (Configuración de empresa)
  DocumentNumbering.cs     (Numeración de documentos)
  SystemParameter.cs       (Parámetro del sistema)
```

8. Application structure (Estructura de aplicación)

```
ERP.Application/
  Abstractions/
  Behaviors/
  Exceptions/
  Interfaces/
  Mappings/
  DTOs/
  Validators/
  Security/
  Companies/
  Catalogs/
  Purchasing/
  Inventory/
  Production/
  Sales/
  Reports/
  Configuration/
  DependencyInjection.cs
```

Organizar cada módulo por caso de uso. No es obligatorio crear carpetas vacías ni un Handler/Validator para cada operación si no se necesitan.

Example use case (Ejemplo de caso de uso)

```
Production/
  CreateProductionOrder/
    CreateProductionOrderCommand.cs
    CreateProductionOrderHandler.cs
    CreateProductionOrderValidator.cs
    CreateProductionOrderResponse.cs
```

Term	Traducción
Command	Comando: solicita una modificación.
Query	Consulta: obtiene información.
Handler	Manejador: ejecuta el caso de uso.
Validator	Validador: verifica condiciones de entrada.

Term	Traducción
Response	Respuesta: datos devueltos por el caso de uso.
DTO (Data Transfer Object)	Objeto de transferencia de datos.

9. CQRS (Separación de comandos y consultas)

```
HTTP Request
    |
    v
Controller / Endpoint
    |
    v
Command or Query
    |
    v
Handler
    |
    +---- Command --> Domain model + persistence
    |
    +---- Query ----> Read projection / query model
    |
    v
Response
```

CQRS no obliga a tener dos bases de datos. En la primera versión se puede usar SQL Server y EF Core para escrituras, y consultas optimizadas con LINQ/proyecciones para lecturas. Separar almacenes de lectura y escritura solo si existe una necesidad medible.

10. Infrastructure structure (Estructura de infraestructura)

```
ERP.Infrastructure/
  Persistence/
    ERPDbContext.cs
    Configurations/
    Migrations/
    Seed/
    DbContextFactory.cs
  Repositories/
  Security/
  Services/
  Cache/
  Logging/
  Email/
  Files/
  Reports/
  Integrations/
  Configuration/
  DependencyInjection.cs
```

Implementar repositorios por agregado cuando aporten valor al modelo de dominio. No es necesario crear un repositorio por cada tabla ni envolver todas las consultas de lectura en repositorios genéricos.

11. API structure (Estructura de la API)

```
ERP.API/
  Controllers/
    AuthController.cs
    UsersController.cs
    CompaniesController.cs
    CustomersController.cs
    SuppliersController.cs
    ProductsController.cs
    InventoryController.cs
    PurchasingController.cs
    ProductionController.cs
    SalesController.cs
    ReportsController.cs
    ConfigurationController.cs
  Middlewares/
  Filters/
  Extensions/
  Security/
  Models/
  Program.cs
  appsettings.json
```

Los endpoints deben mantener un contrato HTTP consistente, validar autenticación y autorización, documentarse con OpenAPI y devolver códigos de estado adecuados. Nunca almacenar contraseñas, claves ni secretos directamente en el repositorio.

12. Data and persistence (Datos y persistencia)

- Usar Entity Framework Core con SQL Server para persistencia relacional.
- Configurar mapeos mediante IEntityTypeConfiguration en carpetas de configuración.
- Gestionar el esquema con migraciones revisadas y versionadas.
- Definir claves, índices, restricciones, precisión decimal y relaciones explícitamente.
- Usar transacciones para operaciones que deban ser atómicas dentro de un mismo contexto de persistencia.
- Auditar operaciones críticas y registrar quién realizó una acción cuando el requisito lo demande.
- No asumir que cada módulo debe tener una base de datos independiente en un monolito modular.

13. Key business flows (Flujos principales)

Purchasing flow (Flujo de compras)

```
Supplier (Proveedor)
  -> Purchase Order (Orden de compra)
  -> Purchase Receipt (Recepción de compra)
  -> Inventory update (Actualización de inventario)
```

Production flow (Flujo de producción)

```
Production Recipe (Receta)
  -> Production Order (Orden de producción)
  -> Material Consumption (Consumo de materiales)
  -> Production Process (Proceso)
  -> Quality Control (Control de calidad, si aplica)
  -> Finished Product (Producto terminado)
  -> Inventory update (Actualización de inventario)
```

Sales flow (Flujo de ventas)

```
Customer (Cliente)
  -> Quotation (Cotización, opcional)
  -> Sales Order (Pedido de venta, opcional)
  -> Sale (Venta)
  -> Invoice (Factura, según requisitos fiscales)
  -> Payment (Pago)
```

Los cambios de inventario deben ser trazables. El módulo responsable registra el movimiento con referencia al documento que lo originó; no se deben modificar existencias sin dejar un historial consistente.

14. Cross-module boundaries (Límites entre módulos)

- Un módulo no debe modificar directamente las tablas internas de otro módulo.
- Preferir interfaces de aplicación, contratos explícitos o eventos de dominio/integración para coordinar comportamientos.
- Mantener la consistencia transaccional dentro de un agregado; evitar agregados enormes que abarquen todo el ERP.
- Para efectos entre módulos que no puedan realizarse en una sola transacción, considerar eventos y procesamiento idempotente.
- Evitar referencias circulares y compartir solo los tipos realmente necesarios.

15. Security and observability (Seguridad y observabilidad)

- Autenticación basada en un mecanismo estándar y validación correcta de tokens.
- Autorización por políticas y permisos; no confiar únicamente en ocultar opciones en la interfaz.
- Almacenar contraseñas usando un algoritmo de hashing apropiado, nunca cifrado reversible ni texto plano.
- Mantener secretos fuera del código y del control de versiones.
- Registrar errores y métricas sin incluir contraseñas, tokens ni datos personales innecesarios.
- Configurar health checks, logging estructurado y manejo uniforme de errores.

16. Testing strategy (Estrategia de pruebas)

Project	Qué validar
ERP.Domain.Tests	Invariantes, cambios de estado, reglas de negocio y eventos.
ERP.Application.Tests	Casos de uso, validaciones, permisos y colaboración con interfaces.
ERP.Infrastructure.Tests	Persistencia, consultas y adaptadores externos.
ERP.API.Tests	Rutas, códigos HTTP, autenticación y formato de respuestas.
ERP.Integration.Tests	Flujos completos con infraestructura de prueba.

Priorizar pruebas de comportamiento y escenarios de negocio, incluyendo casos límite: stock insuficiente, orden cancelada, recepción parcial, merma y finalización de producción.

17. Technology stack (Tecnologías)

Technology	Purpose
.NET 10 / C#	Plataforma y lenguaje de programación.
ASP.NET Core Web API	Exposición de endpoints HTTP.
SQL Server	Base de datos relacional.
Entity Framework Core	Mapeo objeto-relacional y persistencia.
MediatR (opcional)	Despacho de comandos y consultas.
FluentValidation (opcional)	Validación de solicitudes/casos de uso.
OpenAPI / Swagger	Documentación y exploración de la API.
Serilog (opcional)	Logging estructurado.
xUnit	Framework de pruebas.
Moq (opcional)	Dobles de prueba para dependencias.

Confirmar las versiones compatibles de paquetes con .NET 10 al crear la solución. MediatR, AutoMapper y otras bibliotecas deben adoptarse por necesidad y licencia vigente, no por obligación arquitectónica.

18. Naming conventions (Convenciones de nombres)

- Usar inglés para proyectos, namespaces, clases, propiedades, métodos y archivos.
- Usar PascalCase para clases y métodos: ProductionOrder, CreateProductionOrder.
- Usar camelCase para variables y parámetros locales.
- Nombrar operaciones con verbos claros: Create, Update, Cancel, Start, Complete, Get, List.
- Mantener documentación en español con términos ingleses y traducción entre paréntesis cuando ayude al aprendizaje.

```
public class ProductionOrder
{
    public int Id { get; private set; }
    public int ProductId { get; private set; }
    public decimal Quantity { get; private set; }
    public ProductionOrderStatus Status { get; private set; }
}
```

ProductionOrder (Orden de producción) representa una orden para fabricar una cantidad determinada de un producto. El ejemplo es ilustrativo; el modelo definitivo debe proteger invariantes y no permitir cambios de estado inválidos.

19. Development roadmap (Hoja de ruta)

Phase	Scope	Goal
1 — Foundation	Security, Companies, Catalogs	Autenticación, empresas, usuarios y catálogos básicos.
2 — Purchasing & Inventory	Purchasing, Inventory	Órdenes, recepciones, almacenes y movimientos.
3 — Sales	Sales	Cotizaciones, pedidos, ventas y pagos.
4 — Production	Production	Recetas, órdenes, consumos, mermas y producto terminado.
5 — Reports	Reports	Reportes operativos e indicadores.
6 — Advanced features	Multi-branch, multi-currency, integrations	Añadir capacidades solo cuando el negocio las necesite.

20. Architecture decisions (Decisiones arquitectónicas)

Decision	Rationale
Modular monolith first	Reduce complejidad operativa y conserva límites de módulos.
Single relational database initially	Facilita transacciones y despliegue para un ERP pequeño.
CQRS without separate databases	Aporta claridad sin el coste de una infraestructura duplicada.
Pragmatic DDD	Protege reglas relevantes sin sobreingeniería.
English code identifiers	Mantiene convenciones habituales del ecosistema .NET y apoya el aprendizaje del inglés.
Optional technical libraries	Evita acoplar la arquitectura a herramientas que pueden cambiar.

21. Initial implementation checklist (Lista inicial de implementación)

- Crear la solución ERP.sln y los proyectos base.
- Configurar referencias de proyecto siguiendo las reglas de dependencia.
- Configurar DI (Dependency Injection / Inyección de dependencias) y opciones de configuración.
- Configurar SQL Server, DbContext y la primera migración.
- Implementar manejo de errores y Problem Details.
- Crear el primer agregado de dominio y sus pruebas.
- Implementar un caso de uso completo de extremo a extremo.
- Documentar endpoints con OpenAPI.
- Añadir pruebas de integración de persistencia y API.
- Automatizar compilación y pruebas en CI cuando el repositorio esté disponible.

22. Glossary (Glosario)

English	Español
Aggregate Root	Raíz de agregado
Value Object	Objeto de valor
Domain Event	Evento de dominio
Repository	Repositorio
Use Case	Caso de uso
Command	Comando
Query	Consulta
Handler	Manejador
Validation	Validación
Persistence	Persistencia
Dependency Injection	Inyección de dependencias
Warehouse	Almacén
Stock	Existencias
Raw Material	Materia prima
Finished Product	Producto terminado
Waste / Scrap	Merma / desperdicio
Work Center	Centro de trabajo
Audit Trail	Rastro de auditoría
Deployment	Despliegue
Integration Test	Prueba de integración

23. Closing statement (Cierre)

Esta arquitectura es una base inicial y evolutiva para un ERP pequeño, implementado como monolito modular con Clean Architecture, DDD pragmático y CQRS. La estructura exacta debe ajustarse a los requisitos validados, y no todas las carpetas o entidades propuestas necesitan existir desde el primer sprint.

Principio rector: mantener el dominio claro, los casos de uso explícitos, las dependencias controladas y la complejidad proporcional a las necesidades reales del negocio.